An Original SFNNv16-Comparable Multi-Head Tactical Evaluator for Unchessed

Manus AI Research
Unchessed-UCI-EngineClean-room implementation branch: manus/research-facilities

***Abstract*—This paper reports an original, opt-in tactical and terminal evaluator added to the Unchessed chess engine. The implementation is comparable to the supplied SFNNv16 architecture diagrams at the level of public design principles: sparse board-state signals, a reusable existing score, bounded integer-friendly inference, confidence gating, and separated score, terminal, and mate-distance outputs. It does not reproduce Stockfish source code, weights, feature rows, or implementation constants. The default search evaluator is unchanged. The post-change workspace passes 153 tests with six ignored, and a release build plus UCI terminal smoke test succeed. The evidence does not establish numerical SFNNv16 parity or a stable 100/100 result. The remaining requirements are an untouched stratified corpus, independent training, a strict speed gate, and a powered fixed-control match.**

I. Introduction

The research handoff identified a substantial distinction between an ordinary-position calibration result and stable all-position parity. The prior broader-corpus experiment achieved 99.523 cp held-out mean absolute error on a non-mate subset but 155.049 cp on all positions. A direct 120-position comparison reported 422.575 cp mean absolute error, with a 29,887 cp maximum caused by a mate-score mismatch. Consequently, this iteration treats parity as a vector of predeclared gates rather than a filtered scalar claim.

The supplied architecture diagrams describe a modern sparse neural evaluator with perspective-specific inputs, a main subnet, clipped activation functions, and compact output layers. Those diagrams were used only to understand architectural intent. The implementation below is independently authored and uses legal move generation as an explicit tactical feature source. No Stockfish source, weights, or feature rows are included.

II. Clean-Room Boundary and Design

The implementation resides in `unchessed-core/src/tactical_eval.rs`. It receives an existing side-to-move score and computes a sparse tactical state from one legal move list. The feature vector contains legal-move count, checking-move count, forcing-capture count, king-escape count, pinned-piece count, promotion count, low-material status, check status, and a terminal class. Terminal class distinguishes non-terminal, checkmate, and stalemate.

The score head applies a bounded residual:

$$s' = s + \mathrm{clip}(g\,(15C + 6F + 9P + 3K + E), -900, 900), \quad (1)$$

where C, F, P, and K denote checking moves, forcing captures, promotions, and pinned pieces. E is a check-dependent escape-pressure term. The gate g is small for quiet positions, larger for forcing positions, and maximal only for terminal positions. This gate is an original engineering choice; it is not a Stockfish coefficient.

Terminal probability, mate distance, and confidence are emitted as separate heads. This avoids forcing terminal values such as large mate scores into an ordinary static regression output. The path is presentation-only and is exposed through the explicit UCI command `evalbar tactical`. It is not called by the search hot path.

TABLE I
Validation and parity-gate status

Gate	Measurement	Status
Workspace tests	153 passed, 6 ignored	Pass
Focused tactical tests	3 passed	Pass
Release build	Completed	Pass
Checkmate classification	terminal class 1, probability 1000	Pass
All-position MAE	155.049 cp prior result	Open
Search overhead	No new ¡=5% measurement	Open
Powered match	Not run	Open
Stable 100/100 parity	Not established	No claim

III. Implementation Discipline

The editing branch was created from the protected main branch and no main-branch files were edited. The default evaluator and search behavior remain unchanged. The source labels its provenance as an original clean-room research path. The current design intentionally does not implement a recursive proof search or a proof cache; such a mechanism must be benchmarked independently before any playing-evaluator integration.

IV. Validation

Table I summarizes the measured gates. The baseline was frozen before modification at commit `918fccc`. The baseline environment records the compiler, CPU description, kernel, handoff checksum, and source checksums.

The focused tests verify that the start position is non-terminal, checkmate and stalemate are distinct, and a forcing position produces a confidence signal with a bounded score residual. The UCI smoke test on `7k/6Q1/6K1/8/8/8/8/8 b - - 0 1` returned zero legal moves, terminal class 1, mate distance

zero, terminal probability 1000, and the explicit clean-room provenance label.

V. Limitations and Required Next Experiments

The implementation is architecture-comparable, not network-equivalent. Exact numerical parity cannot be inferred from diagrams because the matching trained weights, quantization-aware training recipe, rescored corpus, and evaluation protocol are not present. The next corpus must be game-disjoint and stratified across quiet, checking, capture, promotion, endgame, king-exposed, tactical, and terminal positions. Separate bounded-score, WDL, terminal, and mate-distance targets are required.

The tactical feature extractor currently generates legal moves once per presentation query. This is acceptable for an opt-in evalbar but is not an acceptable ordinary-node search cost without measurement. Before promotion, the team must measure p50/p95/p99 latency, evaluator calls per second, cache hit rate, trigger rate, nodes per second, and overhead against the current approximately 0.8896 nonlinear/HCE ratio. A fixed-control match with sufficient games, balanced colors, deterministic openings, fixed threads and hash, and confidence intervals is also required.

VI. Conclusion

This work provides a maintainable original multi-head tactical foundation that addresses the handoff's central architectural gap: terminal and tactical information is represented separately from ordinary static score. It passes the repository regression gates and demonstrates correct terminal classification in an explicit diagnostic path. It does not achieve stable 100/100 SFNNv16 parity, and the branch deliberately preserves that negative result. A parity claim should be made only after untouched-data, terminal, speed, reliability, and powered-match gates all pass.

References

[1] official-stockfish, "NNUE," Stockfish Documentation. [Online]. Available: https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html

[2] official-stockfish, "UCI Protocol and Stockfish Commands," Stockfish Documentation. [Online]. Available: https://official-stockfish.github.io/docs/stockfish-wiki/UCI-Protocol-and-Stockfish-Commands.html

[3] Stockfish Team, "Stockfish 19," Stockfish Blog. [Online]. Available: https://stockfishchess.org/blog/2026/stockfish-19/